

Importing Builtin Libraries

```
In [1]: import os
```

Loading a Corpus

Using File Method

```
In [2]: filenames = os.listdir("./Data")
```

```
In [3]: corpus = []
for filename in filenames:
    path = os.path.join("Data",filename)
    f = open(path,'r')
    text = f.read()
    corpus.append(text)
    f.close()
```

Using Inbuilt Libraries

```
In [4]: from nltk.corpus import PlaintextCorpusReader
dataset = PlaintextCorpusReader("./Data",".*")
```

Here `".*"` means match all filenames. If we want to match filenames starting with `doc`, we would use `"doc.*"`, and for matching only text files starting with `doc`, we will use `"doc.*\.txt"`.

Therefore:

- `".*"` → all files
- `"doc.*"` → files starting with `"doc"`
- `"doc.*\.txt"` → files starting with `"doc"` and ending with `".txt"`

```
In [5]: corpus = [dataset.raw(file) for file in dataset.fileids()]
```

```
\corpus\reader\api.py:218: RuntimeWarning: Security Warning [CorpusReader]: Path
E:\ML Data Science\Preprocessing\Data\doc9.txt allowed via CWD.
    with self.open(f) as fp:
c:\Users\Gaurish Garg\AppData\Local\Programs\Python\Python312\Lib\site-packages\nltk
\data.py:388: RuntimeWarning: Security Warning [pathsec.open]: Path E:\ML Data Scien
ce\Preprocessing\Data\doc9.txt allowed via CWD.
    stream = _secure_open(self._path, "rb")
```

Preprocessing

Normalisation

```
In [6]: alpha_corpus = []
punctuations = ['.', ',', '!', '?', ';', ':']
for document in corpus:
    document = ''.join(ch for ch in document if ch not in punctuations) ## join wit
    lower_words = [word.lower() for word in document.split()]
    alpha_words = [word for word in lower_words if word.isalpha()]
    normalised_document = ' '.join(alpha_words)
    alpha_corpus.append(normalised_document)
```

What does ' '.join(list) mean?

It is used to **combine a list of strings into a single string**, with a space between each element.

Syntax:

```
'separator'.join(iterable)
```

Tokenization

```
In [10]: tokenised_corpus = []
for document in alpha_corpus:
    words = [word for word in document.split()]
    tokenised_corpus.append(words)
```

Stopword removal

```
In [11]: import nltk
nltk.download('stopwords')
```

```
[nltk_data] Downloading package stopwords to C:\Users\Gaurish
[nltk_data]      Garg\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

Out[11]: True

```
In [13]: from nltk.corpus import stopwords
stop = stopwords.words(fileids='english')
```

```
In [14]: nostopword_corpus = []
for document in tokenised_corpus:
    nostopwords = [word for word in document if word not in stop]
    nostopword_corpus.append(nostopwords)
```

Stemming

```
In [15]: from nltk.stem import PorterStemmer
```

```
In [16]: ps = PorterStemmer()
stemmed_corpus = []
for document in nostopword_corpus:
    stemmed = [ps.stem(word) for word in document if word not in stop]
    stemmed_corpus.append(stemmed)
```

Bag of Words

Bag of Words (BoW)

Definition:

Bag of Words is a text representation technique in Natural Language Processing (NLP) where a document is represented as a collection (bag) of words, ignoring grammar and word order but keeping track of word frequency.

- Each unique word in the corpus forms the **vocabulary**
- Each document is represented as a **vector of word counts**
- Word order is **not considered**

Vocabulary:

[Word1, Word2, Word3, Word4]

Document-Term Matrix:

	Word1	Word2	Word3	Word4
Document 1	1	0	2	1
Document 2	1	0	0	0
Document 3	0	1	1	1

Explanation

- **Document 1** contains:
 - Word1 → 1 time
 - Word3 → 2 times
 - Word4 → 1 time
- **Document 2** contains:
 - Word1 → 1 time
- **Document 3** contains:
 - Word2, Word3, Word4 → each 1 time

Limitations

- Ignores word order and context
- Can create large sparse matrices
- Does not capture semantics

```
In [17]: vocabulary = []
for document in stemmed_corpus:
    vocabulary += document
unique_vocabulary = list(set(vocabulary)) ## set method gets unique vocabulary
```

```
In [18]: bow = []
i = 0
for document in stemmed_corpus:
    counts = {}
    document_id = i
    i += 1
    counts["document_id"] = document_id
    for word in unique_vocabulary:
        counts[word] = document.count(word)
    bow.append(counts)
```

```
In [19]: import pandas as pd
```

```
In [20]: bow_Df = pd.DataFrame(bow)
bow_Df
```

Out[20]:

	document_id	disciplin	benefit	govern	need	healthcar	digit	enhanc	invest	practic
0	0	0	0	0	0	0	0	0	0	0
1	1	0	0	0	1	0	0	0	0	0
2	2	0	0	0	0	0	1	0	0	1
3	3	0	0	0	0	1	0	0	0	0
4	4	0	0	0	0	0	0	0	1	0
5	5	0	0	0	0	0	0	0	0	1
6	6	1	0	0	0	0	0	0	0	0
7	7	0	1	0	0	0	0	1	0	0
8	8	0	0	0	0	1	0	0	0	0
9	9	0	0	1	0	0	0	0	0	0

10 rows × 144 columns



TF-IDF (Term Frequency – Inverse Document Frequency)

Definition:
TF-IDF is a numerical measure that indicates how important a word is to a document in a corpus. It is calculated by multiplying **Term Frequency (TF)** and **Inverse Document Frequency (IDF)**.

Formula

- $TF(t, d)$ = frequency of term t in document d
- $IDF(t) = \log(N / df)$
 - N = total number of documents
 - df = number of documents containing the term
- **$TF-IDF(t, d) = TF(t, d) \times IDF(t)$**

Example

	Doc1_TF	Doc2_TF	Doc3_TF	IDF
Word1	1	0	2	1

	Doc1_TF	Doc2_TF	Doc3_TF	IDF
Word2	1	0	0	0
Word3	0	1	1	1

TF-IDF Matrix

	Doc1_TFIDF	Doc2_TFIDF	Doc3_TFIDF
Word1	1	0	2
Word2	0	0	0
Word3	0	1	1

Interpretation

- Words with **high TF and high IDF** are most important
- Words appearing in all documents get **low IDF (≈ 0)**
- TF-IDF highlights **distinctive words in each document**

```
In [21]: import numpy as np
tf_idf = []
for word in vocabulary:
    features = {}
    features["Word"] = word
    i = 0
    d = 0
    n = len(stemmed_corpus)
    for document in stemmed_corpus:
        feature_name = "TF_"+str(i)
        features[feature_name] = (document.count(word)/len(document))
        i = i+1
        if(word in document):
            d = d+1
    idf = np.log(n/d)+1
    features["IDF"] = idf
    i = 0
    for document in stemmed_corpus:
        term_Frequency_Name = "TF_"+str(i)
        new_Feature_Name = "TF-IDF_"+str(i)
        features[new_Feature_Name] = features[term_Frequency_Name]*idf
        i = i+1
    tf_idf.append(features)
```

```
In [22]: tf_idf_Df = pd.DataFrame(tf_idf)
tf_idf_Df
```

Out[22]:

	Word	TF_0	TF_1	TF_2	TF_3	TF_4	TF_5	TF_6	TF_7	TF_8	...	TF
0	artifici	0.045455	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.150000
1	intellig	0.045455	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.150000
2	transform	0.045455	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.150000
3	modern	0.045455	0.055556	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.118611
4	industri	0.045455	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.150000
...	...	...	...	...	...	...	...	...	...	...	...	...
179	pollut	0.000000	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.000000
180	public	0.000000	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.000000
181	awar	0.000000	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.000000
182	equal	0.000000	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	0.0	...	0.000000
183	import	0.000000	0.000000	0.0	0.0	0.0	0.055556	0.0	0.0	0.0	...	0.000000

184 rows × 22 columns



In [67]:

```
docs = []
for i in range(0,len(stemmed_corpus)):
    word_scores = {}
    word_scores["DocumentID"] = "Document_"+str(i)
    for object in tf_idf:
        word_scores[object["Word"]] = object["TF-IDF_"+str(i)]
    docs.append(word_scores)
tf_idf_df_t = pd.DataFrame(docs)
tf_idf_df_t
```

Out[67]:

	DocumentID	artifici	intellig	transform	modern	industri	machin	learn	r
0	Document_0	0.150118	0.150118	0.150118	0.118611	0.150118	0.118611	0.261312	0.1
1	Document_1	0.000000	0.000000	0.000000	0.144969	0.000000	0.000000	0.000000	0.0
2	Document_2	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.201715	0.0
3	Document_3	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0
4	Document_4	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0
5	Document_5	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0
6	Document_6	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0
7	Document_7	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.212921	0.0
8	Document_8	0.000000	0.000000	0.000000	0.000000	0.000000	0.153496	0.112723	0.1
9	Document_9	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.0

10 rows × 144 columns



Using Inbuilt Functions

```
In [23]: from sklearn.feature_extraction.text import CountVectorizer
```

```
In [24]: base_corpus = [' '.join(document) for document in stemmed_corpus]
```

```
In [25]: cv = CountVectorizer()
output = cv.fit_transform(base_corpus)
```

```
In [26]: bow_ib_df = pd.DataFrame(
        output.toarray(),
        columns=cv.get_feature_names_out()
    )
    bow_ib_df.insert(0, "document_id", [f"Doc_{i}" for i in range(len(base_corpus))])
    bow_ib_df
```

Out[26]:

	document_id	access	across	allow	among	analysi	analyt	analyz	anytim	approach
0	Doc_0	0	0	0	0	0	0	0	0	0
1	Doc_1	0	1	0	0	0	0	0	0	1
2	Doc_2	1	0	1	0	0	0	0	1	0
3	Doc_3	0	0	0	0	1	0	0	0	0
4	Doc_4	0	0	0	0	0	1	0	0	0
5	Doc_5	0	0	0	0	0	0	0	0	0
6	Doc_6	0	0	0	1	0	0	0	0	0
7	Doc_7	0	0	0	0	0	0	0	0	0
8	Doc_8	0	0	0	0	0	0	1	0	0
9	Doc_9	0	0	0	0	0	0	0	0	0

10 rows × 144 columns



Adding a Column using df.insert() in Pandas

Definition

df.insert() is used to **add a new column at a specific position** in a DataFrame.

Syntax

```
df.insert(loc, column, value)
```

```
In [27]: from sklearn.feature_extraction.text import TfidfVectorizer
```

```
In [28]: tv = TfidfVectorizer(norm=None,
        use_idf=True,
        smooth_idf=False,
        sublinear_tf=False)
    output = tv.fit_transform(base_corpus)
```

```
In [29]: tfidf = pd.DataFrame(output.toarray(), columns=tv.get_feature_names_out(output))
    tfidf
```

Out[29]:

	access	across	allow	among	analysi	analyt	analyz	anytim	approach
0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
1	0.000000	3.302585	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	3.302585
2	3.302585	0.000000	3.302585	0.000000	0.000000	0.000000	0.000000	3.302585	0.000000
3	0.000000	0.000000	0.000000	0.000000	3.302585	0.000000	0.000000	0.000000	0.000000
4	0.000000	0.000000	0.000000	0.000000	0.000000	3.302585	0.000000	0.000000	0.000000
5	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
6	0.000000	0.000000	0.000000	3.302585	0.000000	0.000000	0.000000	0.000000	0.000000
7	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
8	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	3.302585	0.000000	0.000000
9	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000

10 rows × 143 columns



Manual TF-IDF vs Sklearn TF-IDF

Observation:

Manual TF-IDF $\approx$ 0.15, while sklearn TF-IDF $\approx$ 3.3. However, $3.3 / 22 \approx 0.15$, showing a clear relationship between the two.

Explanation:

In the manual approach:

$TF = \text{count} / \text{total_words}$, $IDF = \log(n/d)$, so $TF-IDF = (1/22) \times 3.3 \approx 0.15$.

In sklearn:

$TF = \text{raw count}$, $IDF \approx 3.3$, so $TF-IDF = 1 \times 3.3 = 3.3$.

Key Relationship:

$\text{Manual TF-IDF} = \text{Sklearn TF-IDF} / \text{document_length}$

Core Difference:

- Manual method normalizes TF by document length
- Sklearn uses raw counts and may apply normalization later

Conclusion:

Both methods are mathematically correct. The difference arises from how **TF is defined (normalized vs raw count)**, not from an error in computation.

Processing HTML Text for Machine Learning (Using BeautifulSoup)

Objective

To **extract, clean, and convert HTML text into feature vectors** suitable for machine learning.

Step 1: Access HTML Content

HTML data can be obtained from:

- Local `.html` files
- Web pages (via requests)

```
In [30]: import requests
          from bs4 import BeautifulSoup

          url = "https://gaurishgarg.github.io/"

          headers = {
              "User-Agent": "Mozilla/5.0"
          }

          response = requests.get(url, headers=headers)
          html = response.text
          document = BeautifulSoup(html, "html.parser").get_text()
          document
```

[illegible]

```
In [31]: punctuations = ['.', ',', '!', '?', ';', ':', '|', '\\']
document = ''.join(ch for ch in document if ch not in punctuations) ## join without
lower_words = [word.lower() for word in document.split()]
alpha_words = [word for word in lower_words if word.isalpha()]
normalised_document = ' '.join(alpha_words)
normalised document
```

Out[31]: 'gaurish garg about scholastics publications projects skills writing my works at a glance hi gaurish as a master of technology in augmented reality and virtual reality from the indian institute of technology jodhpur silver i work at the intersection of design and engineering to create immersive experiences my focus lies in bridging design and engineering principles so that immersive systems are not only but also technically feasible on end systems contact me download resume my works at a glance scholastics m tech in from iit jodhpur with cgpa silver medalist be in computer engineering from tiet patiala with cgpa passed in unity certified user vr developer exam from unity technologies with score publications fraud alert lessons learned on atm training in rural india view publication spatial audio training for visually impaired users navigation in vr an analytical approach view publication projects wow wizard on watch in vr view project traditional indian games in mixed reality view project precise selection technique for dense ves view project skills a r development ar foundation vuforia vr development meta openxr google xr game development human computer interaction for game logic basic c basic python video editing vinci connect with me through linkedin game design hci gaurish garg all right reserved'

Rest is Same

In []:

In []: